

Introduction to Optimal Control

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 17, 2026

1. Optimal Control Problem (OCP) Formulation

Goal: Find a trajectory $x(\cdot)$ and control input $u(\cdot)$ that minimize a cost function.

Cost Function:

$$J(x, u) = \underbrace{\int_0^T l(x(t), u(t), t) dt}_{\text{Running Cost}} + \underbrace{l_f(x(T))}_{\text{Terminal Cost}}$$

Subject to:

- ① **System Dynamics:** $\dot{x}(t) = f(x(t), u(t), t), \quad \forall t \in [0, T]$
- ② **Initial Condition:** $x(0) = x_0$
- ③ **Path Constraints:** $h(x(t), u(t), t) \leq 0$ (e.g., Torque limits, Joint limits)
- ④ **Terminal Constraints:** $x(T) \in \mathcal{X}_f$ (e.g., $x(T) = 0$)

2. OCP Example: Inverted Pendulum

Let's define an OCP for a simple pendulum swing-up task.

State & Control: $x = [q, \dot{q}]^T$, $u = \tau$

Cost Function (Energy + Effort):

$$\int_0^T (q^2 + \dot{q}^2 + 10^{-3}\tau^2)dt + 10^3 q(T)^2$$

Dynamics:

$$I\ddot{q} = \tau - mgl \sin(q) \implies \frac{d}{dt} \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ I^{-1}(\tau - mgl \sin(q)) \end{bmatrix}$$

Constraints:

- Torque Limit: $|\tau(t)| \leq \tau^{max}$
- Goal: $q(T) \approx 0$ (Upright position)

3. Taxonomy of Optimal Control Methods

How do we solve infinite-dimensional optimization problems?

	Optimize then Discretize (Continuous Time)	Discretize then Optimize (Discrete Time)
Global (Sufficiency)	Hamilton-Jacobi-Bellman (HJB)	Dynamic Programming (DP)
Local (Necessity)	Calculus of Variations / Pontryagin's Max. Principle (PMP)	Direct Methods (NLP, Shooting, Collocation)

- **Indirect Methods:** HJB, PMP (Solve Analytical Equations)
- **Direct Methods:** Transform into NLP (Nonlinear Programming)

4. Dynamic Programming (DP): The Core Idea

Bellman's Principle of Optimality:

"An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

Intuition (Road Trip Example): If the fastest route from Seoul to Busan goes through Daejeon, then the sub-route from Daejeon to Busan must also be the fastest route between those two cities.

- **Key Insight:** We can solve the problem backwards from the end.

5. Discrete-Time DP Formulation

Consider a discrete-time OCP with horizon N :

$$\min_U \sum_{k=0}^{N-1} l(x_k, u_k) + l_f(x_N)$$

subject to $x_{k+1} = f(x_k, u_k)$.

Let's split the cost at step k :

$$\min_{u_k \dots u_{N-1}} \left[l(x_k, u_k) + \underbrace{\sum_{i=k+1}^{N-1} l(x_i, u_i) + l_f(x_N)}_{\text{Cost-to-go from } x_{k+1}} \right]$$

6. The Value Function and Bellman Equation

Define the **Value Function** (Optimal Cost-to-Go) $V_k(x)$:

$$V_k(x) = \text{Min cost to reach target from state } x \text{ at time } k$$

Boundary Condition (Terminal Cost):

$$V_N(x) = I_f(x)$$

Recursive Bellman Equation (Backward Pass):

$$V_k(x) = \min_u [I(x, u) + V_{k+1}(f(x, u))]$$

Optimal Control Policy:

$$u_k^*(x) = \arg \min_u [I(x, u) + V_{k+1}(f(x, u))]$$

7. DP Discussion: Pros and Cons

Advantages:

- Provides a **Global Optimum**.
- Gives a **Feedback Policy** $u^*(x)$ (works for any state).
- Can handle integer variables and complex constraints.

Disadvantages:

- **Curse of Dimensionality:** To implement DP, we must discretize the state space (Grid).
- Example: A 10-DOF robot implies a 20-dimensional state space. A grid with just 10 points per dimension requires 10^{20} points!
- **Result:** Practically limited to low-dimensional systems ($n \leq 4 \sim 6$).

8. From DP to HJB (Continuous Time derivation)

How do we extend DP to continuous time? (Limit $\Delta t \rightarrow 0$)

Discrete Bellman Eq:

$$V(t, x) = \min_u [l(x, u)\Delta t + V(t + \Delta t, x + f(x, u)\Delta t)]$$

Step 1: Taylor Expansion of V

$$V(t + \Delta t, x + f\Delta t) \approx V(t, x) + \frac{\partial V}{\partial t}\Delta t + \nabla_x V \cdot f(x, u)\Delta t$$

Step 2: Substitute back

$$V(t, x) = \min_u \left[l(x, u)\Delta t + V(t, x) + \dot{V}\Delta t + \nabla_x V f(x, u)\Delta t \right]$$

9. The Hamilton-Jacobi-Bellman (HJB) Equation

Step 3: Simplify and Divide by Δt Cancel $V(t, x)$ from both sides and divide by Δt :

$$0 = \min_u \left[l(x, u) + \frac{\partial V}{\partial t} + \nabla_x V f(x, u) \right]$$

Rearranging terms gives the **HJB Equation**:

$$-\frac{\partial V}{\partial t} = \min_u [l(x, u) + \nabla_x V(t, x)^T f(x, u)]$$

Boundary Condition: $V(T, x) = l_f(x)$.

**Note: This is a Partial Differential Equation (PDE). Solving this for high-dim systems is extremely hard.*

10. Pontryagin's Maximum Principle (PMP)

Since solving the HJB PDE is hard, let's look at the gradient of the Value Function along the optimal trajectory.

Define the **Adjoint Variable (Costate)** $\lambda(t)$:

$$\lambda(t) \triangleq \nabla_x V(t, x(t))^T$$

Interpretation: Sensitivity of the optimal cost to perturbations in the state.

Define the **Hamiltonian** H :

$$H(x, u, \lambda) \triangleq l(x, u) + \lambda^T f(x, u)$$

The HJB equation becomes: $-\dot{V} = \min_u H(x, u, \lambda)$.

11. Deriving the Costate Dynamics (Adjoint Equation)

How does λ evolve over time? Let's differentiate the HJB equation w.r.t x .

$$-\nabla_x \left(\frac{\partial V}{\partial t} \right) = \nabla_x \left(\min_u H(x, u, \lambda) \right)$$

Using $\lambda = \nabla_x V$ and assuming u^* minimizes H :

$$-\dot{\lambda} = \nabla_x H(x, u^*, \lambda)$$

$$-\dot{\lambda} = \nabla_x l(x, u^*) + \lambda^T \nabla_x f(x, u^*)$$

Terminal Condition for λ :

$$\lambda(T) = \nabla_x l_f(x(T))$$

12. Summary of PMP Conditions

The PMP converts the optimization problem into a **Boundary Value Problem (BVP)**.

1. State Equation (Forward):

$$\dot{x} = \nabla_{\lambda} H = f(x, u^*)$$

2. Adjoint Equation (Backward):

$$-\dot{\lambda} = \nabla_x H = \nabla_x I + \lambda^T \nabla_x f$$

3. Optimality Condition:

$$u^*(t) = \arg \min_u H(x(t), u, \lambda(t))$$

4. Boundary Conditions:

$$x(0) = x_0, \quad \lambda(T) = \nabla_x I_f(x(T))$$

13. Solving the BVP: Indirect Methods

We have a mixed boundary condition:

- We know $x(0)$, but not $\lambda(0)$.
- We know $\lambda(T)$, but it depends on $x(T)$.

The Shooting Method: Instead of solving BVP directly, treat it as finding the correct initial costate λ_0 .

Function to Solve: Find λ_0 such that the error at the final time is zero.

$$r(\lambda_0) = \lambda(T; \lambda_0) - \nabla_x l_f(x(T; \lambda_0)) = 0$$

where $\lambda(T; \lambda_0)$ denotes integrating forward from (x_0, λ_0) .

14. Single Shooting Algorithm

Algorithm Steps:

- 1 Guess an initial costate λ_0 .
- 2 Integrate the State and Adjoint equations forward from $t = 0$ to $t = T$:

$$\dot{x} = f(x, u^*), \quad \dot{\lambda} = -\nabla_x H$$

where $u^* = \arg \min H$.

- 3 Compute the residual (error) at terminal time:

$$E = \lambda(T) - \nabla_x l_f(x(T))$$

- 4 If $|E| < \epsilon$, Stop.
- 5 Update λ_0 using Newton-Raphson method and Repeat.

15. Newton Step and Sensitivities

To perform the Newton step:

$$\lambda_0 \leftarrow \lambda_0 - (\nabla_{\lambda_0} r)^{-1} r(\lambda_0)$$

We need the Jacobian $\nabla_{\lambda_0} r$ (Sensitivities).

How to compute Sensitivities?

- **Finite Differences:** Perturb λ_0 slightly and re-integrate. (Easy but slow/inaccurate)
- **Sensitivity Differential Equations:** Differentiate the ODEs w.r.t λ_0 and integrate them together. (Accurate but complex derivation)
- **Automatic Differentiation:** Use software (JAX, CasADi) to get gradients.

16. Summary: From Global to Local

- **Dynamic Programming (DP):** Solves for $V(x)$ globally. Breaks down due to Curse of Dimensionality.
- **HJB Equation:** Continuous limit of DP. Still a hard PDE.
- **PMP:** Focuses on the single optimal trajectory using Adjoint variables (λ).
- **Indirect Methods (Shooting):** Turns PMP into a root-finding problem for λ_0 .

Next Lecture: We will explore **Direct Methods** (Collocation, DDP) which discretize the problem first to solve it using nonlinear programming (NLP).